

Mockito

September 21st, 2026

Ana Paiva, Auri Vincenzi, José Campos

Prerequisites

This tutorial introduces the usage of [Mockito](#) for JUnit tests. If you are not familiar with [JUnit](#), please read the [JUnit tutorial](#).

Introduction

[Mockito](#) is a popular open source framework for mocking objects in software testing. Using Mockito greatly simplifies the development of test cases for classes with external dependencies.

Mockito records the interaction with mock and allows you to check if the mock object was used correctly, e.g., if a certain method has been called on the mock. A mock object is a dummy implementation for an interface or a class. It allows defining the output of certain method calls. They typically record the interaction with the system and tests can validate that.

 See Mockito's [limitations](#) for an overview of the current limitations. Note: private methods which are not visible for tests, cannot be mocked. See [Mockito's statement on private methods](#) for details. In any case, we provide a suggestion on how to test private fields or methods [here](#).

Creating mock objects with the Mockito API

Mockito provides several methods to create mock objects:

- Using the `@ExtendWith(MockitoExtension.class)` extension for JUnit 5 in combination with the `@Mock` annotation on fields
- Using the static `mock()` method.
- Using the `@Mock` annotation.

 If you use the `@Mock` annotation, you must trigger the initialization of the annotated fields. The `MockitoExtension` does this by calling the static method `MockitoAnnotations.initMocks(this)`.

For example, consider the following classes `Database` and `Service`.

Java

```
package pt.up.fe;

public class Database {

    public boolean isAvailable() {
        // Currently not implemented, as this is just a demo
        return false;
    }

    public int getUniqueId() {
        return 42;
    }
}
```

Java

```
package pt.up.fe;

public class Service {

    private Database database;

    public Service(Database database) {
        this.database = database;
    }

    public boolean query(String query) {
        return database.isAvailable();
    }

    @Override
    public String toString() {
        return "Using database with id: " +
            String.valueOf(database.getUniqueId());
    }
}
```

A unit test using Mockito which mocks the `Database` object could look like the following:

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class) // Configure Mockito to create the
mocks
// based on the @Mock annotation, this requires JUnit 5, if you are using
an
// older version of JUnit, call Mock.init() in your setup method.
public class ServiceTest {

    @Mock // Configure Mockito to mock the databaseMock instance.
    Database databaseMock;

    @Test
    public void testQuery() {
        assertNotNull(databaseMock);
        // Configure the mock to return true when its isAvailable method is
called.
        when(databaseMock.isAvailable()).thenReturn(true);
        Service service = new Service(databaseMock);
        boolean check = service.query("SELECT * FROM Clients");
        assertTrue(check);
    }
}
```

 Tip: Mockito provides a lot of static methods for mock and asserts. By adding the `org.mockito.Mockito.*`; static import, you can use these methods directly in your tests. Static imports allow you to call static members, i.e., methods and fields of a class directly without specifying the class. Using static imports also greatly improves the readability of your test code.

To run the code above, you need to update `pom.xml` and include the following dependencies:

XML

```
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <version>5.20.0</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.20.0</version>
  <scope>test</scope>
</dependency>
```

Configuring the return values of methods calls on the mock objects

Mockito allows configuring the return values of methods which are called on the mock via a fluent API. Unspecified method calls return "empty" values:

- null for objects
- 0 for numbers
- false for boolean
- empty collections for collections

Using `when().thenReturn()` and `when().thenThrow()`

Mocks can return different values depending on arguments passed into a method. The `when(...).thenReturn(...)` method chain is used to specify a return value for a method call with pre-defined parameters.

You also can use methods like `anyString` or `anyInt` to define that depending on the input type a certain value should be returned. If you specify more than one value, they are returned in the order of specification, until the last one is used. Afterwards the last specified value is returned.

The following demonstrates the usage of `when(...).thenReturn(...)`

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class ServiceDatabaseIdTest {

    @Mock
    private Database databaseMock;

    @Test
    public void ensureMockitoReturnsTheConfiguredValue() {
        // define return value for method getId()
        when(databaseMock.getId()).thenReturn(42);

        Service service = new Service(databaseMock);
        // use mock in test....
        assertEquals(service.toString(), "Using database with id: 42");
    }
}
```

Other examples which demonstrate the configuration of Mockito are in the following

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.ArgumentMatchers.isA;
import static org.mockito.Mockito.when;

import java.util.Iterator;

import org.junit.jupiter.api.Test;
```

```

import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class MockitoWhenExampleTest {

    @Mock
    private Iterator<String> i;

    private Comparable<String> c;

    // demonstrates the return of multiple values
    @Test
    public void testMoreThanOneReturnValue() {
        when(i.next()).thenReturn("Mockito").thenReturn("rocks");
        String result = i.next() + " " + i.next();
        assertEquals("Mockito rocks", result);
    }

    // this test demonstrates how to return values based on the input
    // and that @Mock can also be used for a method parameter
    @Test
    public void testReturnValueDependentOnMethodParameter(@Mock
Comparable<String> c) {
        when(c.compareTo("Mockito")).thenReturn(1);
        when(c.compareTo("Eclipse")).thenReturn(2);
        assertEquals(1, c.compareTo("Mockito"));
        assertEquals(2, c.compareTo("Eclipse"));
    }

    // return a value based on the type of the provide parameter
    @Test
    public void testReturnValueIndependentOnMethodParameter2(@Mock
Comparable<Integer> c) {
        when(c.compareTo(isA(Integer.class))).thenReturn(0);
        assertEquals(0, c.compareTo(Integer.valueOf(4)));
    }
}

```

The `when(...).thenReturn(...)` method chain can be used to throw an exception.

```
Java
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.when;

import java.util.Properties;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MockitoThrowsTest {

    // demonstrates the configuration of a throws with Mockito
    // not a read test, just "testing" Mockito behavior
    @Test
    public void testMockitoThrows() {
        Properties properties = Mockito.mock(Properties.class);
        when(properties.get(Mockito.anyString())).thenThrow(
            new IllegalArgumentException("Stuff"));
        Throwable exception = assertThrows(IllegalArgumentException.class,
            () -> properties.get("A"));
        assertEquals("Stuff", exception.getMessage());
    }
}
```

doReturn when and doThrow when

The `doReturn(...).when(...)` method configuration can be used to configure the reply of a mocked method call. This is similar to `when(...).thenReturn(...)`.

As `when(...).thenReturn(...)` is easier to read, you should prefer using it. But `doReturn` can be useful if you are using spies, e.g., with `@Spy`, in Mockito. In case of a spy, Mockito uses the underlying object and `when(...).thenReturn(...)` will call the underlying method which may create side effects. If these side effects are not desired, use `doReturn`, e.g., if you spy on a list and a certain operation would result in an undesired exception from the original object, like requesting the element in position 2 of an empty list.

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.doReturn;
import static org.mockito.Mockito.spy;

import java.util.Properties;

import org.junit.jupiter.api.Test;

public class MockitoSpyWithListTest {

    // demonstrates of the spy function
    @Test
    public void ensureSpyForListWorks() {
        List<String> list = new ArrayList<String>();
        var spiedList = spy(list);

        doReturn("42").when(spiedList).get(99);
        String value = (String) spiedList.get(99);
        assertEquals("42", value);
    }
}
```

The `doThrow` variant can be used for methods which return `void` to throw an exception. This usage is demonstrated by the following code snippet.

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.doThrow;
import static org.mockito.Mockito.mock;

import java.io.IOException;
import java.io.OutputStream;
import java.io.OutputStreamWriter;
```



```

import org.junit.jupiter.api.Test;

public class MockitoThrowsTest {

    @Test
    public void testForIOException() throws IOException {
        // create an configure mock
        OutputStream mockStream = mock(OutputStream.class);
        doThrow(new IOException()).when(mockStream).close();

        // use mock
        OutputStreamWriter streamWriter = new OutputStreamWriter(mockStream);

        assertThrows(IOException.class, () -> streamWriter.close());
    }
}

```

Using Answers for complex mocks

thenReturn returns a predefined value every time. With an **Answer** object you can calculate a response based on the arguments.

This can be useful if your mocked method is supposed to call a function on one of the arguments or if your method is supposed to return the first argument to allow method chaining. There are different ways to configure an answer:

```

Java
import static org.mockito.AdditionalAnswers.returnsFirstArg;
import static org.mockito.Mockito.doAnswer;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;

@Test
public void answerTest() {
    // with doAnswer():
    doAnswer(returnsFirstArg()).when(list).add(anyString());
    // with thenAnswer():
    when(list.add(anyString())).thenAnswer(returnsFirstArg());
}

```

```
// with then() alias:
when(list.add(anyString())).then(returnsFirstArg());
}
```

Or if you need to do a callback on your argument:

```
Java
List<User> userMap = new ArrayList<>();
 UserDao dao = mock(UserDao.class);
when(dao.save(any(User.class))).thenAnswer(i -> {
    User user = i.getArgument(0);
    userMap.add(user.getId(), user);
    return null;
});
when(dao.find(any(Integer.class))).thenAnswer(i -> {
    int id = i.getArgument(0);
    return userMap.get(id);
});
```

Verify the calls on the mock objects

Mockito keeps track of all the method calls and their parameters to the mock object. You can use the `verify()` method on the mock object to verify that the specified conditions are met. For example, you can verify that a method has been called with certain parameters. This kind of testing is sometimes called behaviour testing. Behaviour testing does not check the result of a method call, but it checks that a method is called with the right parameters.

```
Java
package pt.up.fe;

import static org.mockito.Mockito.atLeast;
import static org.mockito.Mockito.atLeastOnce;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.times;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.verifyNoMoreInteractions;
```

```

import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentMatchers;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class MockitoVerifyTest {

    @Test
    public void testVerify(@Mock Database database) {
        // create and configure mock
        when(database.getId()).thenReturn(43);

        // call method testing on the mock with parameter 12
        database.setId(12);
        database.getId();
        database.getId();

        // now check if method testing was called with the parameter 12
        verify(database).setId(ArgumentMatchers.eq(12));

        // was the method called twice?
        verify(database, times(2)).getId();

        // other alternatives for verifying the number of method calls for a
        method
        verify(database, never()).isAvailable();
        verify(database, never()).setId(13);
        verify(database, atLeastOnce()).setId(12);
        verify(database, atLeast(2)).getId();

        // more options are
        // times(numberOfTimes)
        // atMost(numberOfTimes)
        // This let's you check that no other methods were called on this
        object.
        // You call it after you have verified the expected method calls.

```

```
        verifyNoMoreInteractions(database);
    }
}
```

In case you do not care about the value, use the `anyX`, e.g., `anyInt`, `anyString`, or `any(YourClass.class)` methods.

Wrapping Java objects with Spy

The `@Spy` annotation or the `spy()` method can be used to wrap a real object. Every call, unless specified otherwise, is delegated to the real object. In the example below, the `spy.get(0)` method behaves as specified, but if you call any other method from the `LinkedList` class, it uses the original method's behaviour. For instance, `spy.size()` will return 0 and not 1.

```
Java
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.doReturn;

import java.util.LinkedList;
import java.util.List;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Spy;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class MockitoSpyTest {

    @Spy
    private List<String> spy = new LinkedList<>();

    @BeforeEach
    void setup() {
        // Alternative way of creating a spy
    }
}
```

```

    // List<String> list = new LinkedList<>();
    // List<String> spy = spy(list);
}

@Test
public void testLinkedListSpyCorrect() {
    // when(spy.get(0)).thenReturn("foo");
    // would not work as the delegate it called so spy.get(0)
    // throws IndexOutOfBoundsException (list is still empty)

    // you have to use doReturn()
    doReturn("foo").when(spy).get(0);

    assertEquals("foo", spy.get(0));
}
}

```

Using @InjectMocks for dependency injection via Mockito

You also have the `@InjectMocks` annotation which tries to do constructor, method or field dependency injection of mock objects into other types. It is helpful to wire up objects to be tested.

For example, assume you have a class named `ArticleManager` with the following constructor:

```

Java
package pt.up.fe;

public ArticleManager(User user, ArticleDatabase database) {
    // some code...
}

```

This class can be constructed via Mockito. The parameters to the constructor could be supplied via mock objects as demonstrated by the following code snippet.

Java

```
package pt.up.fe;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class ArticleManagerTest {

    @Mock
    private ArticleDatabase database;

    @Mock
    private User user;

    @InjectMocks // creates an instance of ArticleManager and injects the
    // database mock and the user mock into ArticleManager's constructor
    private ArticleManager manager;

    @Test
    public void shouldDoSomething() {
        // TODO perform some tests with this managers
    }
}
```

⚠ Mockito can inject mocks either via constructor injection, setter injection, or property injection and in this order. So if `ArticleManager` would have a constructor that would only take `User` and setters for both fields, only the mock for `User` would be injected.

Mocking final classes

For example, if you have the following final class:

Java

```
package pt.up.fe;
```

```

public final class FinalClass {
    public final String finalMethod() {
        return "something";
    }
}

```

You can mock it via the following code:

```

Java
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertNotNull;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
public class MockitoMockFinal {

    @Test
    public void testMockFinal(@Mock FinalClass finalMocked) {
        assertNotNull(finalMocked);
    }

    @Test
    public void testMockFinalViaMockStatic() {
        MockedStatic<FinalClass> mockStatic =
Mockito.mockStatic(FinalClass.class);
        assertNotNull(mockStatic);
    }
}

```

Mocking static methods

Mockito also allows you to mock static methods.

Java

```
package pt.up.fe;

public class Utility {
    public static String getDatabaseConnection(String url) {
        return "http://production/" + url;
    }
}
```

You can mock it via the following code:

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;
import org.mockito.Mockito;

public class UtilityTest {

    @Test
    public void shouldMockStaticMethod() {
        try (MockedStatic<Utility> mockedStatic =
            Mockito.mockStatic(Utility.class)) {
            mockedStatic.when(() ->
                Utility.getDatabaseConnection(Mockito.eq("test")))
                .thenReturn("testing");
            mockedStatic.when(() ->
                Utility.getDatabaseConnection(Mockito.eq("prod")))
                .thenReturn("production");

            String result1 = Utility.getDatabaseConnection("test");
            assertEquals("testing", result1);
            String result2 = Utility.getDatabaseConnection("prod");
            assertEquals("production", result2);
        }
    }
}
```



```
}  
}  
}
```

Testing private methods

Essentially, there are two ways to test private methods (assuming we can change the code under test):

(a) indirect testing the private method through a public method that internally invokes the private method, or (b) using traditional/custom made Java reflection to change the visibility of private methods to protected or public.

- Option (a) might be difficult to achieve as the conditions to actually reach the private method might not be easy to setup, e.g., an object that might have to be in a special state.
- Option (b) could be implemented as, for example:

Java

```
// Create an instance of XClass  
XClass xClassInstance = new XClass();  
// Access the private method "foo" using reflection  
Method method = XClass.class.getDeclaredMethod("foo");  
method.setAccessible(true);  
// Invoke the private method "foo"  
method.invoke(xClassInstance);
```

However, given that Java reflection requires much boiler-plate code (to support all Java corner cases) and can be error prone, [PowerMock](#)'s [Whitebox](#) class or [Spring Framework](#)'s [ReflectionTestUtils](#) (which is updated and maintained more often than PowerMock) provide utility methods to ease our testing task.

Let's say that we would like to test the `modifyData` private method of the following class:

Java

```
package pt.up.fe;  
  
public class DataService {
```

```

public boolean replaceData(final String dataId, final byte[] binaryData)
{
    return modifyData(dataId, binaryData);
}

public boolean deleteData(final String dataId) {
    return modifyData(dataId, null);
}

private boolean modifyData(final String dataId, final byte[] binaryData)
{
    // Imagine this method doing something complex and expensive
    return true;
}
}

```

PowerMock alternative

None

```

// Add the following to your pom.xml
<dependency>
    <groupId>org.powermock</groupId>
    <artifactId>powermock-reflect</artifactId>
    <version>2.0.9</version>
    <scope>test</scope>
</dependency>

```

To invoke a private method you can make use of the `Whitebox.invokeMethod(...)` method in [PowerMock's reflect module](#). To do this you simply do:

Java

```

package pt.up.fe;

import org.junit.jupiter.api.Test;
import org.powermock.reflect.Whitebox;

import static org.junit.jupiter.api.Assertions.assertTrue;

```

```

public class DataServiceTest {

    @Test
    public void testModifyData() throws Exception {
        DataService dataService = new DataService();
        boolean result = Whitebox.invokeMethod(dataService,
            "modifyData", "id", new byte[] { 42 });
        assertTrue(result);
    }
}

```

The `Whitebox` class provides a set of methods which could help you bypass encapsulation if it is required. Usually, it is not a good idea to get/modify non-public fields, but sometimes it is only way to cover code by test for future refactoring.

- Use `Whitebox.setInternalState(...)` to set a non-public member of an instance or class.
- Use `Whitebox.getInternalState(...)` to get a non-public member of an instance or class.
- Use `Whitebox.invokeMethod(...)` to invoke a non-public method of an instance or class.
- Use `Whitebox.invokeConstructor(...)` to create an instance of a class with a private constructor.

You may find more examples of PowerMock's bypass encapsulation [here](#).

 Note that [PowerMock](#) used to support mocking of private methods. You may find an example [here](#), which seems to no longer work with JUnit >= 5.

Spring Framework alternative

```

None
// Add the following to your pom.xml
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <version>5.3.4</version>
    <scope>test</scope>
</dependency>

```

To invoke a private method you can make use of the `ReflectionTestUtils.invokeMethod` method in [Spring Framework](#)'s test util module. To do this you simply do:

```
Java
package pt.up.fe;

import org.junit.jupiter.api.Test;
import org.springframework.test.util.ReflectionTestUtils;

import static org.junit.jupiter.api.Assertions.assertTrue;

public class DataServiceTest {

    @Test
    public void testModifyData() throws Exception {
        DataService dataService = new DataService();
        boolean result = ReflectionTestUtils.invokeMethod(dataService,
            "modifyData", "id", new byte[] { 42 });
        assertTrue(result);
    }
}
```

Exercises

1. Get the Java project from [here](#).
2. Extract the downloaded file and open the directory on IntelliJ. At this point you will see a Maven-based Java project with a few classes.

Part I – Using common Mockito's methods

Suppose you are building a simple application with a service that calculates the total price of items in a shopping cart. The service uses a `PriceCalculator` dependency to get the price for each item in the cart. Your task is to use Mockito to mock the behaviour of `PriceCalculator` and write a unit test for the `CartService`.

3. The source code of such an application lives in `pt.up.fe.api`. Explore it.
`pt.up.fe.api.PriceCalculator` This interface provides a method to get the price of an item by its ID.
`pt.up.fe.api.CartService` This class is responsible for calculating the total price of all items in the cart. It relies on `PriceCalculator` to get the price for each item.

4. Test the `CartService` class using Mockito. Complete the test case `testCalculateTotalPrice` in the `CartServiceTest` test class and address the TODOs.

Part II – Using `@Spy` from Mockito

5. Write a test class named `TestingSpy`.
6. Write a test case named `ensureToLearnAboutSpy`.
7. Create a `List<String>` object with the `@Spy` annotation and configure it to return a certain string (e.g., “XPTO”) for the `get(10000000)` on the list.

 Tip: Use `doReturn` to configure the spy.

Part III – Using `@InjectMock`

8. Run the `ArticleManagerTest` on IntelliJ or on the command line with `mvn test`. You should get something like

```
None
org.mockito.exceptions.misusing.InjectMocksException:
Cannot instantiate @InjectMocks field named 'manager' of type 'class
pt.up.fe.inject.ArticleManager'.
You haven't provided the instance at field declaration so I tried to
construct the instance.
However the constructor or the initialization block threw an exception:
null
```

Fix this by providing the required mocks and implementing the TODOs.

Part III – Using Spy and reflection to change private fields

Mockito can not, as of today, mock private fields or methods. Changing private fields or methods during testing should be avoided. Instead, you should try to refactor your code.

Technically you could use a combination of `@Spy` and Java reflection to access private fields or methods.

9. Given the `MyClassWithPrivateFieldAndMethod` class and the test class example in the `MyClassWithPrivateFieldAndMethodTest` class, implement the test case `ensureSpyAndReflectiveAccessCanChangeAPrivateField` which should use reflection to call the private method `meineMethod` and validate that the internal value was changed.

Miscellaneous

- [Java 5 official documentation](#)
- [Mockito official website](#)